

★ WEEK 9 · PROBLEM SET

Finance Applications: Three Hands-On Problems

K-Means by Hand · PCA Eigenvalues · The Noise Edge

Each problem: Statement → Principles & Lab Guide → Hand Calculation → Claude Code Prompt

Small numbers you can compute with a pencil — then a metaprompt that lets Claude Code scale the same idea to real data. Hand first, machine second.

Every Problem Follows the Same Four Steps

Work top to bottom — do not skip the hand calculation, it is where the understanding lives

1 · Statement

the question

A concrete finance scenario with a tiny dataset you can see at a glance.

one slide

2 · Principles

the tools

The formulas involved and a short lab guide: what to compute, in what order.

one slide

3 · Hand Calc

pencil first

Work every number by hand in a table.
Small n , clean arithmetic, no calculator fear.

1–2 slides

4 · Claude Code

scale it up

A metaprompt: paste it into Claude Code and it builds the full lab on real data.

one slide

Hand first, machine second — you check the machine, never the other way around.

Where the Data Comes From

Public / in-script sources first — the prompts fetch free data; CSVs are hand-calc references or offline fallbacks

Problem	Primary source (public · no signup)	Provided fallback file
P1 · K-Means	§ 3b yfinance panel (10 tickers, free)	fml_w9_p1_betas.csv (hand-calc)
P2 · PCA eigenvalues	numbers on the slides — pure pencil work	— (2×2 case, no file needed)
P3 · Noise edge	§ 3b yfinance panel + numpy	fml_w9_panel_offline.csv (offline panel)
Real-data extension	any yfinance ticker list you like	—

Course rule: if data is free online (yfinance, FRED, OpenML) or generated in-script, the prompt uses it.

P1

PROBLEM 1 • K-MEANS

Two Clusters, Found by Hand

Six stocks on a beta line — assign, average, repeat until nothing moves

Six Stocks, One Number Each

Cluster by market beta with $K = 2$. Initial centers: $c_1 = 0.5$, $c_2 = 1.5$

Stock	KO	PEP	JNJ	AAPL	AMZN	META
Market beta	0.5	0.6	0.7	1.3	1.4	1.5

The K-Means loop

- ASSIGN: each stock joins its nearest center.
- UPDATE: each center moves to its group's mean.
- REPEAT until assignments stop changing.

Your deliverables

- The assignment table for iteration 1.
- The updated centers; then iteration 2's check.
- Names for the two clusters a human would give them.

The Tool: Minimize the Total Squared Distance

Assign-then-average is not arbitrary — each move can only shrink J

$$J = \sum_i \min_k (x_i - c_k)^2 \quad \text{assign to the nearest center, then move each center to its group's mean}$$

Why it converges

- Assigning to the nearest center can only lower J .
- Moving a center to its group's mean can only lower J .
- J can't fall forever → the loop must stop. (Where it stops depends on the start — hence `n_init`.)

Lab guide — per iteration

- Step 1: for each stock, compare $|\text{beta} - c_1|$ vs $|\text{beta} - c_2|$.
- Step 2: average each group → new centers.
- Step 3: redo the assignments; stop when nothing moves.

Two Iterations to Convergence

Distances, assignments, means — the whole algorithm in one table

Step	Cluster 1 (defensive)	Cluster 2 (growth)
Iter 1: distances to (0.5, 1.5)	KO 0 · PEP 0.1 · JNJ 0.2 — all nearer c_1	AAPL 0.2 · AMZN 0.1 · META 0 — all nearer c_2
Iter 1: update centers	$c_1 = (0.5+0.6+0.7)/3 = 0.6$	$c_2 = (1.3+1.4+1.5)/3 = 1.4$
Iter 2: re-check nearest	unchanged (JNJ: 0.1 vs 0.7)	unchanged → CONVERGED

$$c_1 = 0.6 \text{ (defensive: KO, PEP, JNJ)} \quad c_2 = 1.4 \text{ (growth: AAPL, AMZN, META)}$$

Converged at (0.6, 1.4) — Defensive vs. Growth

Nobody typed 'sector' anywhere; the structure walked out of the numbers

2 iters

Assignments froze immediately — tiny data, fast loop

0.6

The defensive center:
KO, PEP, JNJ

1.4

The growth center:
AAPL, AMZN, META

n_init=10

Real runs restart from many seeds and keep the best J

What K did here

- K = 2 matched the data's true structure — lucky and lovely.
- K = 3 would split a real group; K = 1 would blur both.

The unsupervised catch

- No F1 exists to prove '2 is right' — the elbow plus judgment decide.
- The machine proposes; the analyst names the clusters.

Scale It: Real Returns, Real Clusters, an Elbow

Paste the metaprompt — Claude Code clusters the live panel and sweeps K

[INPUT] Prompt · Claude Code

```
Build a single Colab-ready Python script that:  
1) downloads the course panel with yfinance: tickers  
   ['AAPL', 'MSFT', 'GOOGL', 'AMZN', 'META', 'KO', 'PEP',  
    'XOM', 'JPM', 'JNJ'], 2020-01-01..2024-12-31,  
   auto_adjust=True; rets = pct_change().dropna();  
   (offline fallback: load fml_w9_panel_offline.csv)  
2) clusters STOCKS: KMeans(n_init=10) on rets.T for  
   K = 2..6; plot inertia vs K and mark the elbow;  
3) for the elbow K, print the cluster membership and  
   propose a name for each group from the tickers;  
4) bonus: cluster DAYS on (market mean return,  
   cross-sectional vol) with K=2 and report what % of  
   days land in the 'storm' cluster;  
5) three-sentence verdict: do the stock clusters match  
   real sectors, in this week's language.
```

[OUTPUT] What to expect

```
elbow near K = 2-3  
defensives (KO, PEP, JNJ)  
  separate from big tech  
XOM / JPM may form their  
  own small cluster  
storm days: a minority,  
  clustered in 2020 & 2022  
  
Your 6-beta walk, run on  
~1,250 real days.
```

P2

PROBLEM 2 · PCA

Eigenvalues You Can Solve

The 2×2 correlation matrix — $1 \pm \rho$, and what the directions mean

Two Tech Stocks, One Correlation

Standardized returns with correlation ρ . Solve PCA completely for $\rho = 0.8$, then $\rho = 0.2$

The correlation matrix is $\Sigma = \begin{bmatrix} 1 & \rho \\ \rho & 1 \end{bmatrix}$. Its eigenvalues and eigenvectors have closed forms — this is the one PCA you solve by hand.

Given

- Case A: $\rho = 0.8$ — the stocks move nearly together.
- Case B: $\rho = 0.2$ — barely linked.
- Total spread (trace) = 2 in both cases.

Your deliverables

- Both eigenvalues for each ρ , and each one's share of the spread.
- The two eigenvector directions and their finance names.
- One sentence: when does compression pay?

The Tool: $1 \pm \rho$ and Two Fixed Directions

For the 2×2 correlation matrix, PCA is a formula, not a computation

$$\Sigma = \begin{pmatrix} 1 & \rho \\ \rho & 1 \end{pmatrix} \Rightarrow \lambda_{1,2} = 1 \pm \rho = \frac{1}{\sqrt{2}}(1, 1) \quad v_2 = \frac{1}{\sqrt{2}}(1, -1)$$

Reading it

- $\lambda_1 = 1 + \rho$ along $(1,1)/\sqrt{2}$: the 'both up together' direction.
- $\lambda_2 = 1 - \rho$ along $(1,-1)/\sqrt{2}$: the 'one vs the other' direction.
- The directions never change — only how the spread divides between them.

Lab guide — per case

- Step 1: $\lambda = 1 \pm \rho$.
- Step 2: shares = $\lambda / 2$ (the trace).
- Step 3: name the directions — market mode and spread mode.

Two Cases Through the Formula

At $\rho = 0.8$ one direction owns 90% of the story; at $\rho = 0.2$ the split is 60/40

	Case A: $\rho = 0.8$	Case B: $\rho = 0.2$
$\lambda_1 = 1 + \rho$ (market mode)	1.8	1.2
$\lambda_2 = 1 - \rho$ (spread mode)	0.2	0.8
Shares of total spread ($\div 2$)	90% / 10%	60% / 40%
Keep only PC1, lose...	10% of the story	40% of the story

$$\rho = 0.8: \lambda = 1.8, 0.2 \quad (90\%/10\%) \quad \rho = 0.2: \lambda = 1.2, 0.8 \quad (60\%/40\%)$$

PC1 Is the Market; PC2 Is the Pairs Trade

And compression pays exactly when correlation is high

90%

One number's share
of two stocks' story at
 $\rho = 0.8$

$(1,1)/\sqrt{2}$

The market mode —
both stocks rise and
fall as one

$(1,-1)/\sqrt{2}$

The spread mode —
long one, short the
other

60/40

At $\rho = 0.2$ compression
barely pays — keep
both

Scaling the intuition

- 500 correlated stocks: PC1 (the market) often carries a huge share.
- The 2×2 case is the whole story, N times over.

The trading reading

- PC2's 'one vs the other' IS a pairs-trade axis.
- Small λ_2 = a tight leash — deviations along it tend to snap back.

Scale It: From 2×2 to the Full Panel

Paste the metaprompt — Claude Code runs PCA on real returns and reads the components

[INPUT] Prompt · Claude Code

```
Build a single Colab-ready Python script that:  
1) downloads the course panel with yfinance (same 10  
   tickers as P1, 2020-2024); rets = pct_change();  
   (offline fallback: fml_w9_panel_offline.csv)  
2) verifies the toy first: build [[1, .8], [.8, 1]],  
   print numpy's eigenvalues, confirm 1.8 and 0.2;  
3) runs PCA on the standardized panel; draws the scree  
   plot (explained variance ratio per component);  
4) prints PC1's loadings per ticker - do they share a  
   sign? - and PC2's loadings - which stocks oppose  
   which?  
5) reconstructs returns from the top 3 PCs and reports  
   the % of variance recovered;  
6) three-sentence verdict naming PC1 and PC2 in this  
   week's market-mode / spread-mode language.
```

[OUTPUT] What to expect

```
toy check: 1.8 / 0.2 exact  
PC1: all loadings same  
     sign - the market mode  
PC2: tech vs defensives,  
     opposite signs  
top 3 PCs recover most of  
     the panel's variance  
  
Your  $1 \pm \rho$  formula,  
wearing 10 tickers.
```

P3

PROBLEM 3 · THE NOISE EDGE

Same Eigenvalue, Opposite Verdicts

$\lambda_+ = (1 + \sqrt{N/T})^2$ on two very different datasets

Two Portfolios Ask the Same Question

Observed eigenvalues include 3.1, 1.5, 1.2, 0.9. Which are signal — and does the answer depend on where you work?

Setting	N (stocks)	T (days)	Question
Course panel (our lab)	10	250	classify 3.1, 1.5, 1.2, 0.9
Big fund universe	500	1,260	would an eigenvalue of 2.5 be signal?

Your deliverables

- λ_+ for both settings (show the arithmetic).
- The signal/noise classification of each observed eigenvalue in the course panel — and the verdict on 2.5 at the big fund.
- One sentence: what do more names or less history do to the edge?

The Tool: the Edge of Pure Randomness

If returns were pure noise, ALL eigenvalues would fall below λ_+ — so anything above it earned its size

$$\lambda_+ = \left(1 + \sqrt{N/T}\right)^2 \quad \text{eigenvalues below } \lambda_+ \text{ are indistinguishable from noise}$$

Reading it

- N/T is the guesses-per-evidence ratio from the lecture.
- Small N/T (few names, long history): tight bulk, low edge.
- Big N/T : the random bulk swells and swallows mid-size eigenvalues.

Lab guide — per setting

- Step 1: $q = N/T$, then $\sqrt{q}$.
- Step 2: $\lambda_+ = (1 + \sqrt{q})^2$.
- Step 3: compare each observed eigenvalue to λ_+ — above = signal.

Two Edges, One Ruler

1.44 for the course panel; 2.66 for the big fund — the ruler moved, the verdicts flipped

	Course panel (10, 250)	Big fund (500, 1260)
$q = N/T$	0.04	0.397
$\sqrt{q}$	0.20	0.63
$\lambda_+ = (1 + \sqrt{q})^2$	$1.2^2 = 1.44$	$1.63^2 = 2.66$
Verdicts	3.1 SIGNAL · 1.5 SIGNAL · 1.2 noise · 0.9 noise	$2.5 < 2.66 \rightarrow$ NOISE at this scale

$$(N=10, T=250): \lambda_+ = 1.2^2 = 1.44 \quad (N=500, T=1260): \lambda_+ = 1.63^2 = 2.66$$

The Edge Rises with N/T — and Humility Rises with It

An eigenvalue is only 'big' relative to what randomness could fake

1.44

Course-panel edge —
3.1 and 1.5 clear it
comfortably

2.66

Big-fund edge — the
same ruler, nearly
doubled

$2.5 \times$

Impressive-looking,
yet inside the random
bulk at 500 names

$2 / 10$

Typical: only a couple
of directions in a
panel are real

The denoising step

- Flatten everything under λ_+ to its average; rebuild the matrix.
- The mirages vanish; the market factor stays.

Week 11's opening line

- Optimizers fed the raw matrix chase the mirages.
- Denoised inputs are why HRP/NCO portfolios survive live trading.

Scale It: Denoise the Panel and Compare Portfolios

Paste the metaprompt — Claude Code draws the bulk, flattens it, and prices the payoff

[INPUT] Prompt · Claude Code

```
Build a single Colab-ready Python script that:  
1) downloads the course panel with yfinance (same 10  
   tickers, 2020-2024); rets = pct_change().dropna();  
   (offline fallback: fml_w9_panel_offline.csv)  
2) computes the correlation matrix, its eigenvalues,  
   and lambda_plus = (1 + sqrt(N/T))**2; plots the  
   eigenvalue histogram with the edge marked;  
3) counts and prints how many eigenvalues are signal;  
4) denoises: flatten sub-edge eigenvalues to their  
   average, rebuild the correlation matrix (renormalize  
   the diagonal to 1);  
5) computes min-variance weights from RAW vs DENOISED  
   matrices and prints them side by side;  
6) three-sentence verdict: which weights are saner and  
   what the histogram showed, in this week's language.
```

[OUTPUT] What to expect

```
edge ~ 1.4-1.5 for the  
10 x ~1250 panel  
1-2 eigenvalues above it  
(PC1 the market, maybe 1)  
raw weights: extreme longs  
and shorts  
denoised: tamer, spread out  
  
Your two-edge table, drawn  
as a histogram.
```

Problem Set — Three Things Your Hands Now Know

You computed every number yourself; the code only repeats what you already understand

1

K-Means: assign, average, converge at (0.6, 1.4)

Two iterations sorted defensives from growth — structure with no labels, and `n_init` for unlucky starts.

2

PCA in closed form: eigenvalues $1 \pm \rho$

At $\rho = 0.8$ the market mode owns 90%; PC2 is the pairs-trade axis. Compression pays when correlation is high.

3

The noise edge moved: 1.44 vs 2.66

The same eigenvalue of 2.5 is signal on our panel and noise at a 500-name fund — N/T sets the ruler.

Hand calculation first — then let Claude Code scale the same idea to real data.